



THE YENEPOYA INSTITUTE OF ARTS SCIENCE COMMERCE AND MANAGEMENT

PROJECT-LOW LEVEL DESIGN

On

Hardened Cloud Infrastructure for Agricultural Data

NAME : Azzam Haris NK
REG NO :23BCCE102

GUIDED BY: Ms. Munisha

TABLE OF CONTENT :

1. Introduction	1
1.1 Purpose and Scope	2
1.2 Project Executive Summary	2
1.3 Constraints	2
1.4 Document Organization	3
2. System Overview	3
3. Module Design	4
3.1 Crypto Engine Module	4
3.2 Key Derivation Module	4
3.3 Vault Storage Module	5
3.4 Secure Delete Module	5
3.5 Web Portal and Dashboard Module	5
4. Data Flow Design	5
5. Database Design	5
5.1 Vault Metadata Table	5
5.2 .agv File Format Table	6
5.3 Function Interface Table	6
6. User Interface Design	6
7. Security Implementation	6
8. Testing and Validation	7
9. Project References	7
10. Glossary	7
11. Front-end	8
12. Back-end	9

1. INTRODUCTION

1.1 Purpose and Scope

Purpose:

This document provides a detailed low-level design of the Hardened Cloud Infrastructure for Agricultural Data system. The goal is to design a system that accepts sensitive agriculture data files, encrypts every file using AES-256-CBC before writing it to disk, stores encrypted blobs in a secure vault, and allows authorised retrieval through password-based decryption with HMAC-SHA256 integrity verification.

The system demonstrates how Data-at-Rest protection works, how cryptographic key derivation prevents brute-force attacks, and how HMAC guarantees tamper detection in a cloud agriculture security context.

Scope:

The system covers:

- AES-256-CBC file encryption and decryption with PKCS7 padding
- PBKDF2-HMAC-SHA256 key derivation with per-file random 256-bit salt
- HMAC-SHA256 integrity seal (Encrypt-then-MAC) on every encrypted file
- Custom .agv binary encrypted file format with embedded metadata
- Flask REST API backend for vault management
- Web portal with drag-and-drop upload, vault dashboard and decrypt modal

In Scope:

- File encryption/decryption via AES-256-CBC with PKCS7 padding
- Password key derivation via PBKDF2-HMAC-SHA256 (310,000 iterations)
- Tamper detection via HMAC-SHA256 (Encrypt-then-MAC)
- Flask-based web portal and REST API at localhost:5000
- Secure deletion with random-byte overwrite before file removal

Out of Scope

- Multi-user authentication and access control
- Cloud object storage (AWS S3, Azure Blob) integration
- Real-time geolocation or IP tracking of upload attempts
- Public cloud deployment and SSL/TLS certificate management

1.2 Project Executive Summary

Agricultural data is increasingly targeted by cyber attackers. Soil reports, crop yield projections, irrigation schedules and proprietary breeding records represent valuable competitive intelligence for farming organisations. Unencrypted cloud storage means a single server breach exposes all data in readable plaintext.

This project introduces a structured encryption-first vault system that:

- **Encrypts every file:** Uses AES-256-CBC encryption with PKCS7 padding before writing to disk

1.3 Constraints

Technical Constraints

- AES key derivation (PBKDF2 with 310,000 iterations) adds 0.3–0.8 seconds per operation — intentional security cost
- vault_metadata.json flat-file storage not suitable for production vaults with thousands of files
- Flask development server single-threaded by default — Gunicorn required for concurrent production use
- Decrypted temporary files in /decrypted_temp/ require a manual cleanup routine in production

Security Constraints

- AES key material must never persist in memory after encryption/decryption completes
- Encryption password must never be stored anywhere — not in cookies, sessions or metadata
- HMAC verification must occur before any decryption bytes are processed
- Vault directory must not be accessible via any public web route

Operational Constraints

- Designed for academic and demonstration purposes on localhost
- Local network testing only — public HTTPS deployment out of scope
- Requires Python 3.10+ and pip packages: cryptography, Flask, Flask-CORS

1.4 Document Organization

The document is structured into:

1. Introduction
2. System Overview
3. Module Design
4. Data Flow
5. Database Structure
6. Interface Design
7. Security Implementation
8. Testing Strategy

2. System Overview

The system is designed using a **layered encryption-first architecture**:

- **Users interact:** Users interact with the web portal to upload files and enter passwords
- **Flask server:** Flask server receives uploads, routes to the crypto engine, and manages vault metadata
- **Crypto engine:** Crypto engine derives AES keys via PBKDF2, encrypts with AES-256-CBC, seals with HMAC-SHA256
- **Encrypted .agv blobs:** Encrypted .agv blobs are written to the encrypted_vault/ directory; metadata logged to vault_metadata.json
- **On retrieval:** On retrieval, HMAC is verified before decryption — wrong password or tampered file returns 401

The system ensures that every uploaded agriculture data file is rendered unreadable before it touches persistent storage, making a storage breach useless to an attacker without the encryption password.

3. Module Design

3.1. Crypto Engine Module

- Implements AES-256-CBC symmetric encryption and decryption using the PyCA cryptography library
- Generates cryptographically random 32-byte salt and 16-byte IV per file using os.urandom()
- Applies PKCS7 padding to plaintext before AES encryption; removes padding after decryption

3.2. Key Derivation Module

- Derives 256-bit AES key from user password and per-file salt using PBKDF2-HMAC-SHA256
- Uses 310,000 iterations following OWASP 2023 minimum recommendation — prevents brute-force
- Re-derives key on every request; key material is never cached or stored

3.3. Vault Storage Module

- Builds custom .agv binary blob:
MAGIC(12)|SALT(32)|IV(16)|HMAC(32)|TIMESTAMP(8)|FNAME_LEN(2)|FNAME|CIPHERTEXT
- Saves encrypted file to encrypted_vault/<uuid>.agv; appends metadata record to vault_metadata.json
- threading.Lock() mutex guards all metadata write operations to prevent concurrent corruption

3.4. Secure Delete Module

- Overwrites .agv file bytes with os.urandom(file_size) before calling os.remove()
- Prevents file content recovery from disk sectors after deletion
- Atomically removes metadata record from vault_metadata.json after secure file wipe

3.5. Web Portal and Dashboard Module

- Flask REST API server binds to port 5000; exposes /api/upload, /api/files, /api/download/:id, /api/delete/:id, /api/stats

- HTML/CSS/JS portal provides drag-and-drop upload, stat cards, encrypted vault table and decrypt modal
- Werkzeug secure_filename() sanitises uploaded filenames to prevent path traversal attacks

4. Data Flow Design

- User selects agriculture data file and enters encryption password on web portal
- Browser sends POST /api/upload with multipart file + password form field
- Flask receives file, saves temporary plaintext to /uploads/ directory
- Crypto engine generates random SALT (32 bytes) + IV (16 bytes) via os.urandom()
- PBKDF2-HMAC-SHA256 (310,000 iterations) derives 256-bit AES key from password + SALT
- AES-256-CBC encrypts plaintext with PKCS7 padding; produces ciphertext
- HMAC-SHA256 computed over IV+Ciphertext (Encrypt-then-MAC pattern)
- Custom .agv blob assembled: MAGIC|SALT|IV|HMAC|TIMESTAMP|FNAME|CIPHERTEXT
- Encrypted .agv saved to encrypted_vault/<uuid>.agv; temp plaintext deleted
- Metadata record appended to vault_metadata.json via thread-safe write
- Flask returns JSON: {file_id, algorithm, original_size, encrypted_size, message}
- Admin views real-time vault stats and file listing on web portal dashboard

5. Database Design

Storage Used: Custom .agv Binary Format + vault_metadata.json (JSON flat file)

Tables Used

5.1 Vault Metadata Table (vault_metadata.json)

Field	Data Type	Description
id	String (UUID4)	Unique file identifier used in all API endpoints
original_filename	String	Original filename of the uploaded agriculture file
original_size	Integer	Size of plaintext file in bytes before encryption
encrypted_size	Integer	Size of AES ciphertext portion of the .agv file
encrypted_path	String	Absolute filesystem path to the .agv encrypted file
algorithm	String	'AES-256-CBC' — encryption algorithm identifier
uploaded_at	Integer (Unix TS)	Seconds since epoch when file was encrypted
status	String	'encrypted' — current lifecycle state of the file

5.2 .agv File Format Table

Field	Size (bytes)	Description
MAGIC_HEADER	12	'AGRIVault_V1' — format version identifier
SALT	32	Random salt for PBKDF2 key derivation
IV	16	Random AES-CBC initialization vector
HMAC	32	HMAC-SHA256 over IV+Ciphertext
TIMESTAMP	8	Unix timestamp (8-byte big-endian uint64)
FNAME_LEN	2	Length of original filename in bytes
ORIG_FILENAME	variable	UTF-8 encoded original filename
CIPHERTEXT	variable	AES-256-CBC encrypted file content

5.3 Function Interface Table

Function	Input	Output	Description
encrypt_file(path,pwd)	str, str	dict	Encrypt file; return result metadata dict
decrypt_file(path,pwd,out)	str,str,str	dict	Verify HMAC; decrypt and save original
derive_key(password,salt)	str, bytes	bytes	PBKDF2-HMAC-SHA256 → 32-byte AES key
compute_hmac(key,data)	bytes,bytes	bytes	Returns 32-byte HMAC-SHA256
get_file_metadata(path)	str	dict	Read .agv header without decryption

6. User Interface Design

The system includes a single web-based interface accessible at <http://127.0.0.1:5000>:

- Secure Vault Web Portal Page with IBM/NCS-branded header
- Drag-and-Drop File Upload Zone with file type icon detection
- Encryption Password + Confirm Password input fields with client-side match validation
- Animated Progress Bar with step labels: Preparing → Uploading → PBKDF2 KDF → Encrypting → Done
- Stat Cards: Encrypted Files count, Original Data size, Encrypted Size, Algorithm label
- Encrypted Vault Table: filename, sizes, algorithm, upload date, Download and Delete buttons
- Decrypt Modal: password input with HMAC error display on wrong password (401 response)
- Toast Notifications: success/error/warning messages top-right on all vault operations

The interface uses a clean agricultural green theme with dark headings, white surface cards and green accent colours throughout.

7. Security Implementation

- AES-256-CBC encryption ensures all agricultural data files are unreadable at rest without the password
- PBKDF2-HMAC-SHA256 with 310,000 iterations makes brute-force password recovery computationally infeasible
- Random 256-bit salt per file prevents rainbow table and precomputation attacks across the vault
- Encrypt-then-MAC (HMAC-SHA256) guarantees tamper detection; HMAC verified before decryption begins
- Constant-time HMAC byte comparison (`_constant_time_compare`) prevents remote timing side-channel attacks
- Original filenames stored inside the encrypted .agv blob — never exposed in filesystem or metadata

8. Testing and Validation

- Roundtrip encryption/decryption tested for small, large, binary and empty files
- Wrong password correctly returns HMAC verification failure (401) without processing any plaintext
- Tampered ciphertext (bit-flip injection) correctly detected by HMAC before decryption attempt
- Two encryptions of identical files produce different ciphertext — confirming IV and salt uniqueness
- Secure delete verified: file bytes overwritten with random data before removal from vault
- Flask API endpoints tested with curl for all 6 routes (upload, files, download, delete, stats, health)

9. Project References

- Python cryptography Library (PyCA) — <https://cryptography.io/en/latest/>
- NIST FIPS 197 — Advanced Encryption Standard (AES)
- OWASP Password Storage Cheat Sheet (2023) — PBKDF2 iteration recommendations
- Flask Official Documentation — <https://flask.palletsprojects.com>
- VS Code IDE — <https://code.visualstudio.com>

10. Glossary

- **AES-256-CBC**: Advanced Encryption Standard with 256-bit key in Cipher Block Chaining mode

- **PBKDF2:** Password-Based Key Derivation Function 2 — stretches passwords into cryptographic keys
- **HMAC:** Hash-based Message Authentication Code — guarantees ciphertext integrity
- **IV:** Initialization Vector — random value ensuring unique ciphertext per encryption
- **Salt:** Random bytes mixed into key derivation to prevent precomputation attacks
- **Flask:** Python micro web framework used for the REST API server
- **Data at Rest:** Data stored on disk or in cloud storage, as opposed to data in transit

11. FRONT-END

The web portal is built with HTML5, CSS3 and Vanilla JavaScript. It provides a Google Drive-style encrypted vault interface with drag-and-drop upload, stat cards, vault table and a decrypt modal. Below are the key frontend code sections shown in VS Code:

```

• • • portal/index.html — Web Portal (Lines 1-35)
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8" /><meta name="viewport" />
5    <title>AgriVault — Secure Agriculture Data Storage</title>
6    <style>
7      :root {
8        --green-500: #22c55e; --green-600: #16a34a;
9        --green-900: #14532d; --bg: #f7f8f5;
10     }
11     .sidebar { width: 240px; background: var(--green-900); }
12     .btn-primary { background: var(--green-600); color: #fff; }
13     .drop-zone { border: 2px dashed var(--green-200); }
14   </style>
15 </head>
16 <body>
17   <div class="layout">
18     <aside class="sidebar">
19       <div class="sidebar-brand">
20         <div class="logo-icon">&#x1f33f;</div>
21         <h1>AgriVault</h1>
22       </div>
23       <nav class="sidebar-nav">
24         <button class="nav-item active">&#x1f512; Secure Vault</button>
25         <button class="nav-item">&#x1f4da; How It Works</button>
26       </nav>
27     </aside>
28     <div class="main">
29       <!-- stat cards, upload zone, vault table -->
30     </div>
31   </div>
32 </body></html>

```

Figure 11.1 — Portal HTML structure and CSS variables (VS Code)

```

• • • portal/index.html — JavaScript Upload Handler (Lines 80-115)
1  async function uploadFile() {
2    if (!selectedFile) { toast('Please select a file first.', 'error'); return; }
3    const pwd = document.getElementById('enc-password').value;
4    const conf = document.getElementById('enc-confirm').value;
5    if (pwd !== conf) { toast('Passwords do not match.', 'error'); return; }
6    const formData = new FormData();
7    formData.append('file', selectedFile);
8    formData.append('password', pwd);
9    showProgress(20, 'Uploading to server...');
10   try {
11     showProgress(40, 'Deriving key (PBKDF2, 310k iter)...');
12     const res = await fetch(API + '/upload', { method: 'POST', body: formData });
13     const data = await res.json();
14     showProgress(100, 'Encrypted successfully!');
15     await loadFiles(); await loadStats();
16   } catch (e) { toast('Error: ' + e.message, 'error'); }
17 }
18
19 async function performDecrypt() {
20   const pwd = document.getElementById('dec-password').value;
21   const res = await fetch(API + '/download/' + currentDecryptId, {
22     method: 'POST', headers: {'Content-Type': 'application/json'},
23     body: JSON.stringify({ password: pwd })
24   });
25   if (res.status === 401) { showDecryptError('Wrong password or tampered file.');
```

```
28   const a    = document.createElement('a');  
29   a.href = url;  a.download = filename;  a.click();  
30 }
```

Figure 11.2 — JavaScript upload and decrypt handlers (VS Code)

12. BACK-END – PYTHON

The backend consists of two Python files: `crypto_engine.py` implementing all AES-256-CBC cryptographic operations, and `app.py` providing the Flask REST API. The full source is shown below in VS Code syntax-highlighted format:

• • • `crypto_engine.py` – AES Encryption Core (Lines 1-42)

```
1 import os, struct, time
2 from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
3 from cryptography.hazmat.primitives import padding, hashes, hmac
4 from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
5 from cryptography.hazmat.backends import default_backend
6
7 MAGIC_HEADER = b"AGRIVault_V1"
8 SALT_SIZE = 32
9 IV_SIZE = 16
10 HMAC_SIZE = 32
11 PBKDF2_ITERATIONS = 310 000 # OWASP 2023 recommended minimum
12
13 def derive_key(password: str, salt: bytes) -> bytes:
14     kdf = PBKDF2HMAC(
15         algorithm = hashes.SHA256(),
16         length = 32, salt=salt,
17         iterations= PBKDF2_ITERATIONS,
18         backend = default_backend()
19     )
20     return kdf.derive(password.encode("utf-8"))
21
22 def encrypt_file(input_path: str, password: str) -> dict:
23     salt = os.urandom(SALT_SIZE); iv = os.urandom(IV_SIZE)
24     key = derive_key(password, salt)
25     with open(input_path, "rb") as f:
26         plaintext = f.read()
27     padder = padding.PKCS7(128).padder()
28     padded = padder.update(plaintext) + padder.finalize()
29     cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
30     encryptor = cipher.encryptor()
31     ciphertext= encryptor.update(padded) + encryptor.finalize()
32     mac = compute_hmac(key, iv + ciphertext)
33     fname_bytes = os.path.basename(input_path).encode("utf-8")
34     fname_len = struct.pack(">H", len(fname_bytes))
35     timestamp = struct.pack(">Q", int(time.time()))
36     blob = (MAGIC_HEADER + salt + iv + mac + timestamp + fname_len + fname_bytes + ciphertext)
37     out_path = input_path + ".agv"
38     with open(out_path, "wb") as f: f.write(blob)
39     return { "encrypted_path": out_path, "algorithm": "AES-256-CBC" }
```

Figure 12.1 — `crypto_engine.py` AES encryption core (VS Code)

• • • `crypto_engine.py` – Decryption & HMAC Verify (Lines 44-85)

```
1 def decrypt_file(encrypted_path: str, password: str, output_dir=None) -> dict:
2     with open(encrypted_path, "rb") as f: blob = f.read()
3     magic_len = len(MAGIC_HEADER)
4     if blob[:magic_len] != MAGIC_HEADER:
5         raise ValueError("Invalid file: not an AgriVault encrypted file.")
6     offset = magic_len
7     salt = blob[offset:offset+SALT_SIZE]; offset += SALT_SIZE
8     iv = blob[offset:offset+IV_SIZE]; offset += IV_SIZE
9     stored_mac = blob[offset:offset+HMAC_SIZE]; offset += HMAC_SIZE
10    timestamp = struct.unpack(">Q", blob[offset:offset+8])[0]; offset += 8
11    fname_len = struct.unpack(">H", blob[offset:offset+2])[0]; offset += 2
12    orig_fname = blob[offset:offset+fname_len].decode("utf-8"); offset += fname_len
13    ciphertext = blob[offset:]
14    key = derive_key(password, salt)
15    expected_mac= compute_hmac(key, iv + ciphertext)
16    if not constant time compare(stored_mac, expected_mac):
17        raise ValueError("HMAC verification failed: wrong password or tampered file.")
18    cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
19    decryptor = cipher.decryptor()
20    padded_p = decryptor.update(ciphertext) + decryptor.finalize()
21    unpadder = padding.PKCS7(128).unpadder()
```

```

22     plaintext = unpadder.update(padded_p) + unpadder.finalize()
23     out_path = os.path.join(out_dir, orig_fname)
24     with open(out_path, "wb") as f: f.write(plaintext)
25     return { "decrypted_path": out_path, "original_filename": orig_fname }
26
27 def _constant_time_compare(a: bytes, b: bytes) -> bool:
28     if len(a) != len(b): return False
29     result = 0
30     for x, y in zip(a, b): result |= x ^ y
31     return result == 0

```

Figure 12.2 — crypto_engine.py decryption with HMAC verify (VS Code)

```

• • •   app.py — Flask REST API Server (Lines 1-60)

1  from flask import Flask, request, jsonify, send_file
2  from flask cors import CORS
3  import os, json, time, uuid, threading
4  from werkzeug.utils import secure_filename
5  from crypto_engine import encrypt_file, decrypt_file
6
7  app = Flask( name ); CORS(app)
8  UPLOAD_FOLDER = "uploads"; ENCRYPTED_FOLDER = "encrypted vault"
9  METADATA_FILE = "vault_metadata.json"; MAX_FILE_SIZE = 100 * 1024 * 1024
10 metadata_lock = threading.Lock()
11
12 @app.route('/api/upload', methods=['POST'])
13 def upload_and_encrypt():
14     if "file" not in request.files:
15         return jsonify({"error": "No file provided"}), 400
16     file = request.files["file"]
17     password = request.form["password"]
18     safe_name = secure_filename(file.filename)
19     temp_path = os.path.join(UPLOAD_FOLDER, safe_name)
20     file.save(temp_path)
21     try:
22         result = encrypt_file(temp_path, password)
23         file_id = str(uuid.uuid4())
24         enc_dest = os.path.join(ENCRYPTED_FOLDER, file_id + ".agv")
25         os.rename(result["encrypted_path"], enc_dest)
26         os.remove(temp_path)
27         return jsonify({"success": True, "file_id": file_id })
28     except Exception as e:
29         return jsonify({"error": str(e)}), 500
30
31 @app.route('/api/download/<file id>', methods=['POST'])
32 def download_and_decrypt(file_id):
33     data = request.get_json()
34     try:
35         result = decrypt_file(entry["encrypted path"], data["password"], DECRYPTED_FOLDER)
36         return send_file(result["decrypted path"], as_attachment=True)
37     except ValueError as e:
38         return jsonify({"error": str(e)}), 401
39
40 if __name__ == "__main__":
41     app.run(debug=True, host="0.0.0.0", port=5000)

```

Figure 12.3 — app.py Flask REST API server (VS Code)